Project 3: Robot Guidance

CS520 - Intro to Artificial Intelligence

Muzzammil Shahe Ansar (fm501)
Harshankar Reddy Gudur (hsg64)
Vishal Singh (vs872)

1
Introduction

1.1 The Grid

We will briefly explain here how grid creation works. The 2D grid needs to have the following properties-

- The size of the grid should be 11 x 11
- The goal should be at (6, 6)
- The goal should always have ways to enter it from all four cardinal directions
- The cells diagonal to the goal should always be blocked
- 10 randomly selected cells must be blocked while keeping in mind all of the above

Since there will be a lot of bookkeeping required for the MDP evaluation, we will create a structure to store everything. First, the code-

```
mutable struct Grid
        size
        γ
        utilities
        policy
        time
        open
        bot_pos
        goal_pos
        crew_pos
        S  # State Space
        S_map # State->Index Map
end
```

We store a few things in this grid. The size is the dimension of the grid. γ stores the discount factor used for policy iteration. Utilities stores the utilities. Policy stores the optimal action given a state. time stores the expected time of a cell. open stores a 2D array which tells us which indices are open and which are closed. The last two lines unfortunately pertain to S and S_map. Unfortunately, Project Overleaf's editor seems not to support the font used for the code. For further details on how the grid structure is populated please refer to **Proj3.jl** in the zip file.

2

No Bot

The "No Bot" scenario is when there's no bot on the ship, just the crew member. The crew member moves randomly (as there's no intervention from the security bot) and eventually finds its way into the teleport pad. This gives us a baseline. The crew member, on their own will find their way to get to the Teleport Pad in some expected time $T_{no\ bot}(cell)$, where $T_{no\ bot}(cell)$ be the expected amount of time it will take the crew member to reach the Teleport Pad , starting at cell and just walking randomly.

2.1 The Random Walk

The movement of the crew member here can be taken as a Random Walk in the Markov Chain. It starts at some given state, and at any state $x_{(i,j)}$ it moves to the next state $x_{(i',j')}$ with some transition probability. The probability for this scenario is evenly divided between every neighbor, making it the same for every action it can take at its current state. The walk ends when the crew member reaches the Teleport Pad .

2.2 The Math

The expected time for the crew member to reach the teleport pad from a particular cell depends on the value of expected time for its neighboring cells. This can be shown mathematically as

$$T_{no\ bot}(cell) = 1 + \left[\sum_{n \in neighbor} P(n) \cdot T_{no\ bot}(n) \right] \tag{2.1}$$

Where P(n) is the transition probability to each neighboring cell (equal among all neighbors that are accessible)

Keep in mind that,

$$T_{no\ bot}(Teleport\ Pad) = 0$$

Solve for the value of $\mathbf{T_{no\ bot}}$ for each cell This is done in code. It can be found below in the section on implementation. The function "grid_time_evaluation_nobot" solves for the answer using the equation above. The equation is repeated till convergence is achieved.

Why dont we need a β? We don't need β since there is no reason to discount the time. β is a factor that allows MDP policies to be affected by future rewards. It prevents our policies from being short-sighted. There is no point in discounting the time here. Time is discrete, and discounting the time of future cells does not make sense. Another thing to keep in mind is that the statespace itself is finite. The purpose of β is to allow us to recursively sum through an infinite state space without having the sum blow up. Not really necessary here.

2.3 Implementation

The implementation of the "No Bot" scenario is trivial. We only really need to know how the crew reacts. The code is-

```
function grid_run_crew(g::Grid)
            # First we need to know if the bot
            # is in the immediate neighbors or not
            crew_neighbors = grid_get_open_crew_neighbors(g, g.crew_pos)

            # If we need to run away-
            if g.bot_pos in crew_neighbors
                    crew_neighbors = filter(x->x != g.bot_pos,
                    ↪  crew_neighbors)
                    future_crew_positions = []
                    for cn in crew_neighbors
                            if length(future_crew_positions) == 0
                                    push!(future_crew_positions, cn)
                            else
                                    d1 = distance(g.bot_pos,
                                            future_crew_positions[length
                                        (future_crew_positions)])
                                    d2 = distance(g.bot_pos, cn)
                                    if d2 > d1
                                            future_crew_positions = [cn]
                                    elseif d2 == d1
                                            push!(future_crew_positions, cn)
                                    end
                            end
                    end
                    g.crew_pos = rand(future_crew_positions)
            # If we are not running away
            else
                    g.crew_pos = rand(crew_neighbors)
            end
    end
```

The process is simple. We first get all the cells neighboring the current crew position. We see if the bot is in the neighborhood (Not possible here since the bot does not

exist). If it is, we flee to the cell that puts the greatest distance between the bot and crew in one move. If not, we randomly choose one of the neighbors to move to. This latter behavior is what is seen in the no bot scenario.
The game is run using the following bit of code-

```
function grid_run_sim(g::Grid, bot_type::BotType, params=nothing, k_max=500)
        for turn in 1:k_max
                grid_run_bot(g, bot_type, params)
                grid_run_crew(g)
                if g.crew_pos == g.goal_pos
                        return (true, turn)
                        break
                end
        end
        return (false, k_max)
    end
```

We run the game for a maximum of 500 turns. The bottype chooses which bot to use. In this case we use NO_BOT. The game is simple. We first execute the bot (which is ignored for the NO_BOT type). We then execute the logic for the crew from above. We then check for the termination condition. If we do terminate, we return true and the turn at which termination was achieved. If not, we return false and k_max. As for evaluation of expected time, the following bits of code is used The code for this is -

```
function grid_lookahead_time_nobot(g::Grid, crew_pos)
        if crew_pos == g.goal_pos
                return 0
        end
        accessible_crew_pos = grid_get_open_crew_neighbors(g, crew_pos)
        # If we are at the goal position, time will be 0
        # If not,
        t = 1 # Every accessibe action is 1 turn away
        for crew_pos′ in accessible_crew_pos
                t += g.time[g.S_map[crew_pos′]] *
                ↪  1.0/length(accessible_crew_pos)
        end
        return t
    end

    # Time Evaluation No bot
    function grid_time_evaluation_nobot(g::Grid, k_max = 1000)
        # Make a few changes since there is no bot
        # State space is the position of crew alone
        g.S = [(i, j) for i in 1:g.size for j in 1:g.size]
        # Filter out blocked cells from state space
        g.S = filter(x->g.open[x[2]][x[1]], g.S)
        g.S_map = Dict(s=>i for (i, s) in enumerate(g.S))
        g.time = [0 for s in g.S]
```

```
            for k in 1:k_max
                if DEBUG == 3
                    println("Time Evaluation: $k")
                end
                time = [grid_lookahead_time_nobot(g, s) for (i, s) in enumerate(g.S)]
                if sum(abs.(time - g.time))/length(time) < 1e-1
                    println("Convergence acquired!")
                    break
                end
                g.time = time
            end
        end
```

The lookahead function implements equation (2.1). We simply iterate through all accessible crew positions and calculate the time for them by using our equation. The second function evaluates the lookahead equation multiple times across all possible crew member positions. It also make some changes to the grid to evaluate the time. Once convergence is achieved, we know the expected steps to victory for each cell.

To take the average over all cells (Asked in a section after this), we use the following bit of code-

```
        # Change this grid_time_evaluation for optimal bot evaluation
        # Change this grid_time_evaluation_nobot for no bot evaluation
            grid_time_evaluation_nobot(grid)
        println(grid.time)
        # Expected value of T_no_bot when crew_pos is chosen at random
        println("Expected Time to victory if crew is chosen randomly: $(mean(grid.time))")
```

Here, it simply prints the mean of the result from the time evaluation function.

2.4 Expected Time

If the crew member is initially randomly chosen cell (uniform), what is the expected time to get to the Teleport Pad ? If we assume that the crew member cell is randomly chosen, then the expected time is simply the average over all open cells in the grid. This can be done numerically. We iterate on all the values of T_nobot until we reach convergence. Once we have converged, we simply take the mean over all crew_positions. Please look above for implementation details. We generally get a value of about 190 steps, depending on the structure of the grid.

```
Expected Time to victory if crew is chosen randomly: 186.25266354777952
```

Figure 2.1: Expected steps to victory if crew cell is chosen randomly

Are there any cells with an infinite expected time?
Imagine a case where our blocked cells form a closed area. In such a case, the $T_{no\ bot}(cell)$ for all the cells in that closed area will have an infinite waiting time, because there is no escape. Aside from the above scenario, there are no cells that have an infinite expected time. We did not find any in all of our experiments so far.

3

Optimal Bot

In this scenario, the aim is to considerably decrease the time required for the crew member to reach the teleport pad by utilizing an optimal bot programmed to take strategic measures in efficiently guiding the crew member; where the bot actively steers the path of the crew member, employing deliberate actions to direct them towards the teleport pad.

3.1 The Math

Let T_{bot}*(bot cell, crew cell)* be the expected amount of time until the crew member enters the teleport pad, *if the bot is taking optimal actions to herd the crew member into the teleport pad.*

$$T_{bot}(bot\ cell,\ crew\ cell) = 1 + \arg\min_{a\ \in\ action} \left[\sum_{c'\ \in\ crewStates} P(c'|c, b, a) \cdot T_{bot}(b',\ c') \right] \tag{3.1}$$

Where $P(c'|c,\ a)$ is the probability of a crew member moving to state c' given the current state 'c' and an action 'a' (NORTH, SOUTH, EAST, WEST, NORTHEAST, NORTHWEST, SOUTHEAST, SOUTHWEST, STAY).

Solve for the value of $\mathbf{T_{bot}}$ for each cell pair, and note the optimal action to take in each case We use a utility approach to extract the policy that gives us the lowest amount of time. We then jump off of it to calculate the value of T_{bot} for each cell pair. The code for that is-

```
function grid_lookahead_time(g::Grid, s, a)
  accessible_states = grid_get_accessible_states(g, s, a)
  # If we are at the goal position, time will be 0
  if s[2] == g.goal_pos
    return 0
  end
  # If not,
  t = 1 # Every accessibe action is 1 turn away
  for ´s in accessible_states
    if DEBUG == 1
      println("´s in accessible_states")
    end
    if ´s ∉ g.
      println("[ERROR] Can't find state in statespace!")
```

```
      println("Relevant data: $accessible_states")
    end
    t += g.time[g.._map[´s]] *  1.0/length(accessible_states)
  end
  return t
end

function grid_time_evaluation(g::Grid, k_max = 100)
  for k in 1:k_max
    if DEBUG == 3
      println("Time Evaluation: $k")
    end
    time = [grid_lookahead_time(g, s, g.policy[i]) for (i, s) in enumerate(g..)]
    if sum(abs.(time - g.time))/length(time) < 1e-1
      println("Convergence acquired!")
      break
    end
    g.time = time
  end
end
```

It essentially calls the lookahead equation repeatedly till the results converge. That allows us to get a good idea of T_bot.

3.2 Implementation

For our optimal bot, we have some initial utility value set. We basically iterate through the grid (as many times as it takes) updating our policy each time. We stop doing this once our new policy (the one we got by iterating through the grid another time) is identical to our previous policy. Let's break down how we do this.

We iterate through the grid multiple times, and in each iteration we do the following:

1. We run our `grid_policy_evaluation()` function. This function adjusts the utility for every state of the grid. The function is as follows:

```
function grid_policy_evaluation(g::Grid, k_max = 5)
    for k in 1:k_max
        g.utilities = [grid_lookahead(g, s, g.policy[i]) for (i, s) in enumerate(g
    .S)]
    end
end
```

   We can see here that the `grid_policy_evaluation()` function calls our `grid_lookahead()` function 5 times. This is done for faster policy convergence. But what happens in this second function that we call?

   We go through all the states we can access from our current state, and enumerate through them to determine the utility of our current state using the equation for value iteration. This function returns the new utility for a given state, which is then stored in our grid's *utility* list.

```
function grid_lookahead(g::Grid, s, a)
    accessible_states = grid_get_accessible_states(g, s, a)
    u = grid_reward_function(g, s, a)
```

```
    t_prob = 1.0 / length(accessible_states)

    for s' in accessible_states
        if s' ∉ g.𝒮
            println("[ERROR] Can't find state in statespace!")
            println("Relevant data: $accessible_states")
        end

        u += g.γ * g.utilities[g.𝒮_map[s']] * t_prob
    end

    return u
end
```

2. We get our new policy by going through every state in the grid and getting the best action for the respective state.

3. Compare the previous policy *g.policy* and our new policy. If they're both the same, we break out of the loop. Otherwise we continue with the next iteration.

The entire process runs in our `grid_policy_iteration()` function as shown below:

```
function grid_policy_iteration(g::Grid, k_max = 19)
    for k in 1:k_max
        grid_policy_evaluation(g)
        new_policy = [grid_get_best_action(g, s)[2] for s in g.𝒮]

        if all([new_policy[i] == g.policy[i]
                    for i in 1:length(g.policy)])
            break
        end

        g.policy = new_policy
    end
end
```

This function converges in around 12-15 iterations.
A few helper functions are also important to understand. The most important of these are- "grid_get_accessible_states" and "grid_get_best_action". Lets start with the first-

```
    function grid_get_accessible_states(g::Grid, s, a::Action)
    crew_pos= s[2]
    bot_pos = s[1]
    future_bot_pos = action_map[a] .+ bot_pos
    crew_neighbors = grid_get_open_crew_neighbors(g, crew_pos)
    # If the action makes the bot go into an invalid spot, we return an empty list
    if !grid_is_valid_index(g, future_bot_pos) ||
      !grid_is_open(g, future_bot_pos)
      return []
    end

    # If an action leads to overlapping we return an empty list
    if future_bot_pos == crew_pos
      return []
```

```
  end
  # Let us see if the bot will be detected by the crew
  if future_bot_pos in crew_neighbors
    if DEBUG == 1
      println("[DEBUG] Crew will run!")
    end
    # Crew member will detect the future bot position
    # It will now run to the farthest neighbor
    # So, we first find all the neighbors that put the crew
    # the farthest away from  the future bot position
    crew_neighbors = filter(x->x != future_bot_pos, crew_neighbors)
    future_crew_positions = []
    for cn in crew_neighbors
      if length(future_crew_positions) == 0
        push!(future_crew_positions, cn)
      else
        d1 = distance(future_bot_pos,
          future_crew_positions[length(future_crew_positions)])
        d2 = distance(future_bot_pos, cn)
        if d2 > d1
          future_crew_positions = [cn]
        elseif d2 == d1
          push!(future_crew_positions, cn)
        end
      end
    end
    # We now use that knowledge to construct the possible future states
    accessible_states = [(future_bot_pos, future_crew_pos) for
    future_crew_pos in future_crew_positions]
    return accessible_states
  else
    if DEBUG == 1
      println("[DEBUG] Crew will act normally")
    end
    # Filter out invalid crew positions
    #future_crew_pos = filter(x->grid_is_valid_index(g, x), future_crew_pos)
    # Crew member will not detect the future bot
    accessible_states = [(future_bot_pos, future_crew_pos) for
    future_crew_pos in crew_neighbors]
    return accessible_states
  end
end
```

This is a very long function, but it does model the behavior of the different entities on the grid, so we must go through it. The function first applies the given action to derive the future bot position. It also gets all the neighboring cells of the current crew cell. We first make sure if the conditions are valid - the future bot position should be within bounds and open, and not overlap with the crew. If not, we return an empty list. Now we know what one part of the state (The bot position) will be. We need to understand the second part too. We first see if the future bot position will be in the crew member's vicinity. If it is, the crew member will run away. He will choose the farthest neighbor accessible by one crew move. If not, he will randomly move into one of the available neighbors.
Lets go through the second function now-

```
  function grid_get_best_action(g::Grid, s)
  possible_actions = grid_get_valid_actions(g, s)
  if length(possible_actions) == 0
    println("SHOULD NOT HAPPE")
    println("Relevant Variables:")
    println("State: $s")
    println("Actions: $possible_actions")
    error("Action Length is 0")
  end
  if DEBUG == 1
    print("Possible Actions: $possible_actions")
  end
  u, a = findmax(a->grid_lookahead(g, s, a), possible_actions)
  if a == nothing
    error("No detected Action!")
  end
  return (u, possible_actions[a])
end
```

We start off by getting all valid actions we can execute given our current state. We then calculate the lookahead function for every single action. Which ever results in the highest lookahead utility value will be the optimal action. We find the action and return it and the utility associated with it.

Are there any pairs where the optimal action for the bot is to just stay still? Yes. The following is an output from one random run of the program provided-

```
Better to execute STAY for all these states: Any[((1, 1), (1, 2)), ((1, 1), (2, 1)), ((1, 2), (1, 3)), ((1, 7), (1, 6)), ((1, 8), (1, 7)), ((1, 9), (1, 8)), ((1, 10), (1, 9)
, ((1, 10), (2, 10)), ((2, 1), (1, 1)), ((2, 1), (2, 2)), ((2, 1), (3, 1)), ((2, 2), (2, 3)), ((2, 2), (3, 2)), ((2, 3), (1, 2)), ((2, 3), (2, 4)), ((2, 4), (2, 5)), ((2, 5)
 (2, 6)), ((2, 5), (3, 5)), ((2, 6), (2, 5)), ((2, 6), (3, 6)), ((2, 7), (2, 6)), ((2, 7), (4, 7)), ((2, 8), (2, 7)), ((2, 8), (3, 8)), ((2, 9), (2, 8)), ((2, 9), (3, 9)), (
2, 10), (1, 10)), ((2, 10), (2, 9)), ((2, 10), (2, 11)), ((2, 10), (3, 10)), ((2, 11), (3, 11)), ((3, 1), (3, 2)), ((3, 1), (4, 1)), ((3, 2), (2, 1)), ((3, 2), (3, 3)), ((3,
2), (4, 2)), ((3, 3), (2, 2)), ((3, 3), (3, 4)), ((3, 4), (2, 3)), ((3, 4), (3, 5)), ((3, 4), (4, 4)), ((3, 5), (2, 4)), ((3, 5), (3, 6)), ((3, 5), (4, 5)), ((3, 6), (4, 6)),
 ((3, 10), (4, 10)), ((3, 11), (4, 11)), ((4, 1), (4, 2)), ((4, 1), (5, 1)), ((4, 2), (3, 1)), ((4, 2), (4, 3)), ((4, 3), (3, 2)), ((4, 3), (4, 4)), ((4, 3), (5, 3)), ((4, 4
, (3, 3)), ((4, 4), (4, 5)), ((4, 5), (3, 4)), ((4, 5), (4, 6)), ((4, 6), (3, 5)), ((4, 6), (5, 6)), ((4, 6), (6, 6)), ((4, 7), (3, 6)), ((4, 8), (4, 7)), ((4, 9), (4, 8)), (
(4, 10), (4, 9)), ((4, 11), (5, 11)), ((5, 1), (4, 1)), ((5, 1), (6, 1)), ((5, 3), (4, 2)), ((5, 3), (5, 4)), ((5, 4), (4, 3)), ((5, 4), (6, 6)), ((5, 6), (4, 5)), ((5, 6), (
7, 6)), ((5, 8), (7, 6)), ((5, 9), (5, 8)), ((5, 10), (5, 9)), ((5, 11), (5, 10)), ((5, 11), (6, 11)), ((6, 1), (6, 2)), ((6, 2), (5, 1)), ((6, 2), (6, 3)), ((6, 2), (7, 1)),
 ((6, 3), (6, 4)), ((6, 4), (5, 3)), ((6, 4), (6, 5)), ((6, 4), (7, 3)), ((6, 5), (4, 7)), ((6, 5), (6, 7)), ((6, 7), (7, 8)), ((6, 7), (8, 6)), ((6, 8), (6, 7)), ((6, 8), (7
, 9)), ((6, 9), (6, 8)), ((6, 9), (7, 10)), ((6, 10), (6, 9)), ((6, 11), (5, 11)), ((6, 11), (6, 10)), ((6, 11), (7, 11)), ((7, 1), (7, 2)), ((7, 2), (6, 1)), ((7, 2), (7, 3
), ((7, 2), (8, 1)), ((7, 2), (8, 3)), ((7, 3), (7, 4)), ((7, 4), (8, 3)), ((7, 8), (8, 7)), ((7, 8), (8, 9)), ((7, 9), (7, 8)), ((7, 9), (8, 10)), ((7, 10), (6, 10)), ((7, 
0), (7, 9)), ((7, 10), (8, 11)), ((7, 11), (6, 11)), ((7, 11), (7, 10)), ((8, 1), (7, 1)), ((8, 3), (7, 1)), ((8, 3), (7, 2)), ((8, 3), (8, 4)), ((8, 3), (9, 2)), ((8, 4), (9
, 3)), ((8, 4), (9, 5)), ((8, 6), (7, 6)), ((8, 6), (9, 5)), ((8, 7), (9, 8)), ((8, 9), (7, 9)), ((8, 10), (7, 10)), ((8, 10), (9, 11)), ((8, 11), (7, 11)), ((9, 1), (8, 1)),
 ((9, 3), (8, 3)), ((9, 3), (10, 2)), ((9, 5), (8, 5)), ((9, 6), (8, 6)), ((9, 6), (10, 5)), ((9, 8), (10, 9)), ((9, 9), (8, 9)), ((9, 9), (9, 10)), ((9, 10), (8, 10)), ((9,
10), (10, 9)), ((9, 10), (10, 11)), ((9, 11), (8, 11)), ((9, 11), (9, 10)), ((10, 1), (9, 1)), ((10, 2), (9, 2)), ((10, 2), (10, 3)), ((10, 3), (9, 3)), ((10, 3), (10, 4)),
(10, 3), (11, 2)), ((10, 4), (8, 5)), ((10, 4), (11, 3)), ((10, 5), (9, 5)), ((10, 5), (11, 4)), ((10, 5), (11, 6)), ((10, 6), (9, 6)), ((10, 6), (11, 7)), ((10, 8), (9, 8))
 ((10, 8), (10, 7)), ((10, 8), (11, 9)), ((10, 9), (9, 9)), ((10, 9), (10, 8)), ((10, 9), (11, 10)), ((10, 11), (9, 11)), ((11, 1), (10, 1)), ((11, 1), (11, 2)), ((11, 2), (
0, 2)), ((11, 2), (11, 1)), ((11, 2), (11, 3)), ((11, 3), (11, 4)), ((11, 4), (11, 5)), ((11, 5), (10, 5)), ((11, 6), (10, 6)), ((11, 6), (11, 5)), ((11, 7), (10, 7)), ((11,
7), (11, 6)), ((11, 8), (10, 8)), ((11, 8), (11, 7)), ((11, 9), (10, 9)), ((11, 9), (11, 8)), ((11, 10), (11, 9)), ((11, 10), (11, 11)), ((11, 11), (10, 11)), ((11, 11), (11
 10))]
```

Figure 3.1: States where staying is the optimal move

3.3 T_{bot} vs $T_{no\ bot}$

How do the time to escape values for the no-bot situation compare with time to escape with the bot? How much improvement is there, and what configurations give very little improvement?

The time to escape greatly decreases with the intervention of the Optimal Bot. Relative to the case where the crew member just moves about randomly, the movement of the bot on the ship now helps us "guide" the crew member to a location that we prefer. Doing so decreases the chance of the crew member moving to areas in the grid which would increase the expected time for it to reach the Teleport Pad. The

number of steps decrease from about 200 in the case of "No Bot" to an average of 32 steps when run empirically.

There is very little improvement in the configurations when the crew member is close to the Teleport Pad . In such cases the crew member is already very close to the teleport pad. In both scenarios ("No Bot" and "Optimal Bot") the crew member would reach the Teleport Pad in a few steps.

3.4 Ideal Position

The ideal position of the bot can be found out by finding the bot position for which the expected time for the crew to reach the goal is the lowest. One output of a random run is shown in the figure here. The most optimal choice is (4, 7) for this particular grid. Generally regardless of the grid, the ideal position will concentrate around the middle of the grid, which also makes logical sense.

```
Expected Time to Victory as function of bot position: Dict{Any, Any}((11, 8) => 26.830617676532768, (11, 10) => 27.051283339425385, (11, 2) => 28.78515940391952, (7, 1) => 27
.6213251049007 3, (4, 6) => 25.58382156737128, (9, 3) => 27.237554474137454, (7, 8) => 25.559045902044442, (5, 11) => 26.181384263364432, (7, 10) => 25.770872927932196, (8, 9)
 => 25.727597351009717, (7, 2) => 27.057520189006414, (4, 7) => 25.426049972370205, (1, 9) => 26.57075525962751, (2, 1) => 27.087795937708325, (10, 1) => 28.706425028024544,
(6, 5) => 25.869152686704354, (2, 8) => 25.953487798581243, (10, 8) => 26.439400644747 26, (2, 10) => 26.484760072833605, (6, 11) => 26.09706465981648, (2, 2) => 26.7145773236
9452, (3, 9) => 25.885433579346103, (10, 2) => 28.167497837522937, (11, 6) => 27.186821132215044, (11, 7) => 26.996787594030167, (7, 6) => 25.970185177236722, (9, 1) => 28.33
805235431725 2, (4, 5) => 25.58367370102174 5, (8, 3) => 26.819186226281627, (4, 11) => 26.263717443489956, (9, 8) => 25.984484482540555, (1, 3) => 26.77934910898189, (8, 4) =>
 26.437733586296822, (9, 10) => 26.210389119058494, (5, 9) => 25.57909385310929, (9, 2) => 27.789761793142112, (2, 6) => 25.87296273445651, (10, 6) => 26.6525574257295, (3, 3
) => 26.20713134396663, (3, 4) => 25.949743951924678, (2, 7) => 25.764731952570443, (10, 7) => 26.486234062665 59, (6, 9) => 25.479255129933126, (11, 5) => 27.49970540040206,
(11, 11) => 27.298956268513525, (8, 1) => 27.996040259230845, (9, 6) => 26.388232461926712, (1, 1) => 27.42932206909563, (7, 11) => 26.1800987746797, (5, 3) => 26.18172412947
038, (4, 9) => 25.643237863847244, (1, 8) => 26.29237875431507, (5, 4) => 25.8381492604806, (1, 10) => 26.80269814654021, (8, 10) => 25.961104557577222, (1, 2) => 27.05874246
0654145, (3, 1) => 26.97293917823932, (2, 5) => 25.951057749543203, (10, 5) => 26.939333220853648, (2, 11) => 26.827943053828093, (10, 11) => 26.835000242990205, (6, 3) => 26
.203868713806088, (3, 8) => 25.619940596679335, (3, 10) => 26.15769025497181, (6, 4) => 25.947311144427353, (3, 2) => 26.520791485466468, (11, 9) => 26.83111968338891, (8, 6)
 => 26.132026419363246, (5, 1) => 27.101352908700623, (9, 5) => 26.626482280807604, (1, 6) => 26.210816466805305, (4, 3) => 26.072459755449124, (9, 11) => 26.596618455214266,
 (4, 4) => 25.831739923685888, (5, 8) => 25.435716914284484, (7, 9) => 25.649452324061134, (5, 10) => 25.76297292814942, (8, 7) => 26.078604784060236, (1, 7) => 26.1925059029
53563, (3, 6) => 25.610649803981172, (6, 1) => 27.244089693035296, (2, 9) => 26.10561884149797, (10, 9) => 26.356113655414863, (6, 8) => 25.580442109455092, (6, 10) => 25.778
089265730745, (6, 2) => 26.739816485725353, (11, 3) => 28.20546145414842, (11, 4) => 27.854600252337846, (4, 1) => 26.99819740079031, (5, 6) => 25.692089786384198, (8, 5) =>
26.263222535067992, (7, 3) => 26.532045768958323, (8, 11) => 26.30206470334333, (4, 8) => 25.552946705665185, (7, 4) => 26.12929468480012, (4, 10) => 25.959426532845974, (9,
9) => 26.082277087009654, (4, 2) => 26.434547040182 2, (6, 6) => 25.949118992971616, (2, 3) => 26.38620462029334, (3, 5) => 25.750383812002447, (10, 3) => 27.559305620732108,
(2, 4) => 26.157654210133 52, (3, 11) => 26.565207290409, (10, 4) => 27.261263854314 24, (6, 7) => 25.686316790507796, (11, 1) => 29.36882919725675)
Most optimal position for bot placement: (4, 7) => 25.426049972370205 steps
```

Figure 3.2: Ideal Position

How does the time to escape compare in this case to the randomly placed crew situation previously? With this into consideration, the time to escape will be relatively lower on an average when compared to the randomly placed crew situation previously.

4
Learned Bot

Here, we train a model that takes as input a bot/crew configuration, and gives as output the optimal action for the bot to take in that particular configuration.

4.1 Our Model

As mentioned, we code this project mainly in Julia. Julia has it's own machine learning library called Flux, which is what we utilize in order to create our model.

4.1.1 Structure

We first start off with the data preprocessing. We iterate over the state space and store the flattened state in the vector s'. We append this vector s' to an array X and the corresponding action for that state to Y_raw. We use the 'onehotbatch' function to convert the actions in Y_raw to one-hot encoded vectors, converting it to format suitable for training a neural network. At this point, normally one would split the data into three parts - training data, testing data and validation data. Seeing how we only have a limited amount of data owing to the fixed grid size, it would be best to use it all to train the model. The natural question that comes up is - What do we do about overfitting? How do we know the model works outside of the training data? The answer is simple - when there is 0 chance of data outside of the training model being inputted to the model, why worry about it? Overfitting here can serve us. Unlike the generalized model we will be developing later, for this model, overfitting is not of concern. So, we choose to use the entire dataset as training data. We then use a Data Loader to choose a batch size, which in our case is 64, but it changes often depending on the number of epochs and how much of a hurry I am in.

```
begin
  function learn_policy(g::Grid)
    # Data Preprocessing
    X = []
    Y_raw = []
    for (i, s) in enumerate(g.𝒮)
      s' = [s[1][1], s[1][2], s[2][1], s[2][2]]
      if length(X) == 0
        X = s'
      else
        X = hcat(X, s')
      end
```

```
    push!(Y_raw, g.policy[i])
  end
  action_list = [NORTH, SOUTH, WEST, EAST, NORTHEAST, NORTHWEST,
  SOUTHEAST, SOUTHWEST, STAY]
  Y = Flux.onehotbatch(Y_raw, action_list) # Convert to one hot encoding
  if DEBUG == 3
    println("X: $(size(X))")
    println("Y: $(size(Y))")
  end
  # Choosing batch size
  loader = Flux.DataLoader((X, Y), batchsize=64, shuffle=true)
```

To create the actual model, we use the function `Chain` within the Flux library. We use `4 dense layers`, (with 4, 50, 40, 30 nodes, and finally the output) where essentially each neuron connects to every single neuron of the preceding layer, using `ReLU` (Rectified Linear Unit) activation functions for each layer, which introduces non-linearity and solves the vanishing gradient problem (as opposed to say using the sigmoid function). We then have a final layer of the `softmax` activation function, converting the raw outputs of the neural networks into vector probabilities. Neural networks are very powerful and are capable of learning a vast array of mappings given enough time and data.

4.1.2 Training the Model

To train the model, we use the `ADAM Optimizer` and set the learning rate to 0.001 (after testing various values for the learning rate, 0.001 seemed to perform the best). We train our model over a `2000 epochs`, iterating over each of the batches created in the data loader before. For each batch, we compute the loss using the current model parameters, perform backpropagation to compute the gradients, and update the model parameters using the optimizer, while recording the loss values in the 'loss_history' array. The loss function we used is, of course, `Cross-Entropy`, given by:

$$Loss = -\sum_{c \in Classes} y_c * log(\hat{y_c}) \tag{4.1}$$

Where y_c is the actual action and $\hat{y_c}$ is the predicted action.

We then calculate the delta loss (difference between the first and last loss values) followed by calculating the `accuracy`, by comparing the predicted actions of the model with the ground truth actions from Y_raw. We use the function onecold to convert y from a one-hot encoded vector back to a feature (actions).

```
  # Model Creation
  model = Flux.Chain(
    Flux.Dense(4 => 50, Flux.relu),   # 4 Input Nodes for 2x2D coordinates
  Flux.Dense(50 => 40, Flux.relu),
    Flux.Dense(40 => 30, Flux.relu),
  Flux.Dense(30=>9),# 9 Actions are available
    Flux.softmax)

  # Model Training
  # Optimizer setup
  optim = Flux.setup(Flux.Adam(0.001), model)
```

```
    loss_history = []
    for epoch in 1:1000
      for (x, y) in loader
        loss, grads = Flux.withgradient(model) do m
          y' = m(x)
          Flux.crossentropy(y', y)
        end
        Flux.update!(optim, model, grads[1])
        push!(loss_history, loss)
      end
    end
    println("Delta Loss = $(loss_history[1] -
    loss_history[length(loss_history)])")
    println("Accuracy = $(count(Flux.onecold(model(X), action_list) .== Y_raw)/size(X)
    [2])")
    return model
  end
  model = learn_policy(grid)
end
```

```
The Accuracy of our model comes out to be ~90%
```

4.2 Overfitting

Overfitting is an issue that arises when the model used fits too closely to the training data, so much so that the model struggles to make accurate predictions. A pattern to observe overfitting is to see the model's performance on the training set continuing to improve, while it's performance on a validation set improves to a certain point and then starts to get worse. At this point, you're sure that you're overfitting the data. This is usually an issue when the model is too complex. To combat this, we can use dropout layers. Of course, for this particular bot, there is no point in preventing overfitting. Overfitting is only an issue if the model will see datapoints it has not previously seen. Since we are training it over our entire state space, even if it does overfit, it works in our favor.

Does accuracy of your model translate to effectiveness of a bot based on your model (i.e., can it herd the crew member to the teleport)? Experiment, compare, and verify. Explain your results. Accuracy is important. While the difference between 85% and 90% does not result in any significant difference (observed empirically), the difference between 70% and 90% is quite huge. Our current model performs a bit worse than the optimal model but compared to the no bot baseline it is still a big improvement. It also doesn't have to deal with the memory consumption of MDP structures. Look at the figure for a comparison made by running a game 100 times with all 3 bots- As can be seen, with just 90% accuracy, the learned bot performs almost as well as the optimal bot.

Is your model smaller or larger in memory than the lookup table you computed in the previous section? The size of the lookup-table on disk is 214 times the size of our model in the memory, and hence our model's memory is much

```
Delta Loss = 2.1112485
Accuracy = 0.8956974078645742
Optimal Bot Results:
Average:33.52
StdDev: 1.9715212400580417
Learned Bot Results:
Average:34.82
StdDev: 2.355435416223506
No Bot Results:
Average:199.55
StdDev: 15.358661237230281
```

Figure 4.1: Simulation Results

smaller than the previously computed lookup table.

```
julia> ratio = Base.summarysize(g)/Base.summarysize(model)
214.45639164756813
```

Figure 4.2: Size Comparison

5
Generalizing

Here, we build and train a model capable of guiding the bot regardless of what ship configuration it is on. Here we use Python with PyTorch because of GPU issues. Flux would not run on our laptops on our GPUs. There is an identical model available in Julia for use as well. For the time being, we will explain the Python version.

5.1 Approach

How are you approaching this problem?
Here, the idea is to build a neural network that can learn the best action to take for any state in any instance of the grid. To this end, we train our model across multiple distinct grids with varying bot and crew member configurations. The way we do this is simple. Previously we only gave the state as the input. Here we give the blocked cells along with the state as input. This results in a 32 element input vector. We then pass that through a bigger neral network. We must also be sure not to overfit this neural network. We do not have the luxuries we had in the previous bot. This model will see data it wasn't tested on.

Where are you getting your data from?
We generate multiple grids with random crew and bot configurations and use the data we obtain from those grids to feed into our generalized model. Along with the grid, we also generate the optimal policy for each respective grid as the output (using the method explained in Chapter 3). The grid data as input along with these policies as the output are what we use for training our model.
We generated a total of 80 grids (and their respective optimal policies) to use for training, testing, and validation.

```
function gather_data(k_max=16)
    Threads.@threads for k in 1:k_max
      println("[TID:$(Threads.threadid())}] Gathering Turn: $k")
      g = create_grid()
      print_grid(g)
      grid_policy_iteration(g)
      grid_store_policy(g, "grid$k.file")
    end
```

```
function grid_policy_iteration(g::Grid, k_max = 19)
    for k in 1:k_max
      if DEBUG == 3
```

```
        println("Policy Iteration: $k")
      end
      grid_policy_evaluation(g)
      new_policy = [grid_get_best_action(g, s)[2] for s in g.𝒮]
      if all([new_policy[i] == g.policy[i] for i in 1:length(g.policy)])
        if DEBUG == 1
          println("Early convergence!")
          println("Convergence Iter: $k")
          println([new_policy[i] == g.policy[i] for i in 1:length(g.policy)])
        end
        break
      end
      g.policy = new_policy
    end
  end
```

5.2 Model

We build our model with an input layer, three hidden layers and an output layer as shown below. The number of cells in our hidden layers were decided through trial and error.
We use `5 Linear layers`, where essentially each neuron connects to every single neuron of the preceding layer, using `LeakyReLU` (Leaky Rectified Linear Unit) activation functions for each layer, which introduces non-linearity and solves the dying ReLU problem (where some neurons just become completely inactive during training). We then have a final layer of the `Softmax` activation function, converting the raw outputs of the neural networks into vector probabilities. For training the model, we give as

input the state space and the cells that are blocked. For each state space $\mathcal{S}$, there are 4 integers (as each position is represented as a pair of coordinates, and we have one bot and one crew member coordinate). Additionally, there are 14 blocked cells in total, all of them represented by a total of 28 integers. Combined, that's an input size of 32.

Our output is an action 'a', which can be one of 9 possible movement option our bot has (NORTH, SOUTH, EAST, WEST, NORTHEAST, NORTHWEST, SOUTHEAST, SOUTHWEST, STAY) which gives us an output size of 9.

Knowing that our input is 32 integers and our output is any of 9 possible options, we have designed our model as below:

```
if MODEL == "LEARNED":
    model = nn.Sequential(
        nn.Linear(4, 50),
        nn.ReLU(),
        nn.Linear(50, 40),
        nn.ReLU(),
        nn.Linear(40, 30),
        nn.ReLU(),
        nn.Linear(30, 9),
        nn.Sigmoid())
elif MODEL == "GENERAL":
```

```
    model = nn.Sequential(
        nn.Linear(32, 70),
        nn.LeakyReLU(),
        nn.Linear(70, 80),
        nn.LeakyReLU(),
        nn.Linear(80, 40),
        nn.LeakyReLU(),
        nn.Linear(40, 30),
        nn.LeakyReLU(),
        nn.Linear(30, 9),
        nn.Softmax(dim=1))
else:
```

5.3 Training and Testing

As mentioned before, we have 80 unique grids (and policies) that are going to be used for training and testing this model. We take these 80 and divide them into training, testing and validation datasets. This is how we combat overfitting. The second the validation loss starts increasing, we have started overfitting. It also allows us to evaluate exactly where the overfitting starts and when this happens for different architectures

66.7% of our grids are going to be used for training, 29.97% for testing, and the remaining 3.33% for validation. That basically means 53 grids for training, 24 for testing and 3 for validation. We split our training, testing and validation data as below:

```
    X_train, X_test, y_train, y_test =
            train_test_split(X, y, test_size=0.33)
    X_test, X_val, y_test, y_val =
            train_test_split(X_test, y_test, test_size=0.1)

    X = torch.tensor(X_train, dtype=torch.float32)
    y = torch.tensor(y_train, dtype=torch.float32)

    X_test = torch.tensor(X_test, dtype=torch.float32)
    y_test = torch.tensor(y_test, dtype=torch.float32)

    X_val = torch.tensor(X_val, dtype=torch.float32)
    y_val = torch.tensor(y_val, dtype=torch.float32)
```

Using our `X_train` and `y_train`, we train our model for 2000 epochs with a batch size of 8192. We use `'zero_grad'` to get rid of the gradients before our back propagation, as the accumulated gradients (old gradients) interfere with the new gradients. The loss function used is the `BCELoss` (Binary Cross Entropy Loss) as it proves effective in measuring the error of reconstruction in an auto-encoder.

```
epochs = 2000
batch_size = 8192

for epoch in range(epochs):
    for i in range(0, len(X), batch_size):
```

```
        Xb = X[i:i+batch_size]
        y_hat = model(Xb)
        yb = y[i:i+batch_size]
        loss = loss_fn(y_hat, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

5.4 Results

We have already previously established that a high accuracy (80% here) will result in good performance. This model will not perform as well as the learned bot or the optimal bot. Regardless, it should perform really well given the accuracy.

```
Accuracy 0.8137760940952257
y_true_labels argmax: [2 2 0 ... 5 5 6]
              precision    recall  f1-score   support

           0       0.79      0.76      0.77     25844
           1       0.85      0.86      0.85     48525
           2       0.78      0.77      0.78     25233
           3       0.81      0.81      0.81     38627
           4       0.79      0.80      0.80     27645
           5       0.77      0.78      0.77     25452
           6       0.78      0.77      0.78     26098
           8       0.82      0.78      0.80      3756

    accuracy                           0.80    221180
   macro avg       0.80      0.79      0.80    221180
weighted avg       0.80      0.80      0.80    221180
```

Figure 5.1: Classification Report

The Accuracy for this particular generalized model comes out to be: 80%. Again, it makes logical sense why the accuracy of this model, trained and tested over 80 different grids, would be less accurate than a bot trained and tested on the same one grid. The average number of steps in this particular scenario (obtained over 100 runs) comes out to be around 38 steps.

```
Optimal Bot Results:
Average: 26.35
StdDev: 1.6106110879367466
Learned Bot Results:
Average: 29.47
StdDev: 3.412800941850008
General Bot Results:
Average: 34.68
StdDev: 4.6372347436338694
No Bot Results:
Average: 199.81
StdDev: 18.971847202921076
```

Figure 5.2: Comparison of all bots

6
Bonus: Alien

The state space will grow to higher dimension. The alien position is now also a part of it. We also measure a new metric for this: the failure rate. If the crew gets killed, it is considered a failure.

6.1 Optimal Bot w Alien

To work with an alien, we must first modify a few functions. There are many trivial modifications (Such as simply adding alien position to the state). We shall not cover those. If interested please have a look at the code file provided. We will cover the most important functions though. We will start with what is perhaps the most important function in MDPs - the reward function.

```
function grid_reward_function(g::Grid, state, a)
  accessible_states = grid_get_accessible_states(g, state, a)

  reward = 0
  for accessible_state in accessible_states
    if accessible_state[3] == accessible_state[2]
      return -3.0/length(accessible_states)
    elseif accessible_state[2] == g.goal_pos
      reward += 1.0/length(accessible_states)
    end
  end
  return reward
end
```

We now have a new branch. Whenever the alien is in the same position as the crew, the crew will die. We want to avoid this situation as much as possible. It is a terrible position to be in. We set it to -3.0. The reason for the higher value compared to the reward for reaching the goal position is to encode a sense of risk aversion. It is better to reach the pad slightly late if we survive more often. Of course, this ends up increasing the average turns to victory but as shall see our failure rate remains very low.
The other important function we must go through is our model of the crew's and alien's movements-

```
  function grid_get_accessible_states(g::Grid, s, a::Action)
  alien_pos = s[3]
  crew_pos= s[2]
  bot_pos = s[1]
  future_bot_pos = action_map[a] .+ bot_pos
```

```
    crew_neighbors = grid_get_open_crew_neighbors(g, crew_pos)
    alien_neighbors = grid_get_open_crew_neighbors(g, alien_pos)
    accessible_states = []
    # If the action makes the bot go into an invalid spot, we return an empty list
    if !grid_is_valid_index(g, future_bot_pos) ||
      !grid_is_open(g, future_bot_pos)
      return []
    end

    # If an action leads to overlapping we return an empty list
    if future_bot_pos == crew_pos || future_bot_pos == alien_pos
      return []
    end
    # Let us see if the bot will be detected by the crew
    if future_bot_pos in crew_neighbors
      if DEBUG == 1
        println("[DEBUG] Crew will run!")
      end
      # Crew member will detect the future bot position
      # It will now run to the farthest neighbor
      # So, we first find all the neighbors that put the crew
      # the farthest away from  the future bot position
      crew_neighbors = filter(x->x != future_bot_pos, crew_neighbors)
      future_crew_positions = []
      for cn in crew_neighbors
        if length(future_crew_positions) == 0
          push!(future_crew_positions, cn)
        else
          d1 = distance(future_bot_pos,
            future_crew_positions[length(future_crew_positions)])
          d2 = distance(future_bot_pos, cn)
          if d2 > d1
            future_crew_positions = [cn]
          elseif d2 == d1
            push!(future_crew_positions, cn)
          end
        end
      end
      # We now use that knowledge to construct the possible future states
      accessible_states = [(future_bot_pos, future_crew_pos) for
      future_crew_pos in future_crew_positions]
    else
      if DEBUG == 1
        println("[DEBUG] Crew will act normally")
      end
      # Filter out invalid crew positions
      #future_crew_pos = filter(x->grid_is_valid_index(g, x), future_crew_pos)
      # Crew member will not detect the future bot
      accessible_states = [(future_bot_pos, future_crew_pos) for
      future_crew_pos in crew_neighbors]
    end
    #Similar behavior is expected from the alien
    if future_bot_pos in alien_neighbors
      # The alien will now proceed to run away
      alien_neighbors = filter(x->x != future_bot_pos, alien_neighbors)
      future_alien_positions = []
```

```
    for an in alien_neighbors
      if length(future_alien_positions) == 0
        push!(future_alien_positions, an)
      else
        d1 = distance(future_bot_pos,
          future_alien_positions[length(future_alien_positions)])
        d2 = distance(future_bot_pos, an)
        if d2 > d1
          future_alien_positions = [an]
        elseif d2 == d1
          push!(future_alien_positions, an)
        end
      end
    end
    accessible_states = [(future_bot_pos, future_crew_pos, future_alien_pos) for (_,
  future_crew_pos) in accessible_states for future_alien_pos in future_alien_positions
  ]
  else
    # Choose from any of the alien neighbors

    accessible_states = [(future_bot_pos, future_crew_pos, future_alien_pos) for (_,
  future_crew_pos) in accessible_states for future_alien_pos in alien_neighbors]
  end
  return accessible_states
end
```

The addition here is another section modeling the alien. It is very similar to how the crew member works. If the alien is in the immediate vicinity of our bot along the cardinal directions, it will choose the move that will put it farthest away from the bot. As can be seen, it is implemented very similarly to the crew members. Once this is done, we combine it with the previously derived future crew positions and the future bot position. This gives us the entire list of the accessible states. The rest of the functions are mostly similar except for a few minor modifications. The code for the alien itself is-

```
   function grid_run_alien(g::Grid)
      alien_neighbors = grid_get_open_crew_neighbors(g, g.alien_pos)

      if g.bot_pos in alien_neighbors
          # Run away
    alien_neighbors = filter(x->x != g.bot_pos, alien_neighbors)
    future_alien_positions = []
    for an in alien_neighbors
      if length(future_alien_positions) == 0
        push!(future_alien_positions, an)
      else
        d1 = distance(g.bot_pos,
          future_alien_positions[length(future_alien_positions)])
        d2 = distance(g.bot_pos, an)
        if d2 > d1
          future_alien_positions = [an]
        elseif d2 == d1
          push!(future_alien_positions, an)
        end
      end
```

```
            end
            g.alien_pos = rand(future_alien_positions)
        else
            g.alien_pos = rand(alien_neighbors)
            # Proceed as normal
        end
    end
```

The alien behaves almost exactly like the crew. It will randomly move about but if it comes close to the bot, it will flee.

6.2 Learned Bot w Alien

In this case, the main difference from the previous implementation is that we now also include the alien position in the input layer. This can be seen here-

```
        ...
            ´s = [s[1][1], s[1][2], s[2][1], s[2][2], s[3][1], s[3][2]]
        ...
      model = Flux.Chain(
      Flux.Dense(6 => 64, Flux.relu),   # 6 Input Nodes for 3x2D coordinates
    Flux.Dense(53 => 32, Flux.relu),
      Flux.Dense(32 => 32, Flux.relu),
    Flux.Dense(32=>9),# 9 Actions are available
      Flux.softmax)
```

We can get similarly high values of accuracy (90%) with this model for the learned bot. The epochs do need to be set higher for training though. The learned bot works well and was nearly as effective as the one without alien.

6.3 Generalized Scenario w Alien

Again, similar to the previous situation, the dimensions need to be changed to include the new alien position. The structures were not changed otherwise. We got 80% accuracy as before. We saw degradation in both failure rate and average steps compared to the previous two bots. The code is -

```
      # Model Creation
    model = Flux.Chain(
        Flux.Dense(34 => 70, Flux.leakyrelu),   # 34 Input Nodes for 2x2D coordinates
      Flux.Dense(70 => 80, Flux.leakyrelu),
      Flux.Dense(80 => 40, Flux.leakyrelu),
      Flux.Dense(40 => 30, Flux.leakyrelu),
    Flux.Dense(30=>9),# 9 Actions are available
      Flux.softmax)
```

6.4 Results

These results are a bit worse than not having aliens, but compared to the baseline of not having a bot, and the upper bound of the optimal bot, the two learned bots

```
Optimal Bot Results:
Failure Rate: 0.01
Average: 36.61
StdDev: 2.5071198106996255
Learned Bot Results:
Failure Rate: 0.02
Average: 40.13
StdDev: 4.121581759013363
General Bot Results:
Failure Rate: 0.05
Average: 45.85
StdDev: 3.177599642247638
No Bot Results:
Failure Rate: 0.21
Average: 198.32
StdDev: 17.949357863380957
```

Figure 6.1: Alien Results

performed admirably. The average step count is quite good, and the failure rates are much lower than the 21% of the no bot scenario.

7
Contributions

The project was equally contributed to by all three members. Every step from brainstorming ideas to implementing code was done together.